



Amazon Kinesis

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to Kinesis



Amazon Kinesis is the AWS fully managed solution for **collecting, processing, and analyzing streaming data** in the cloud.

When you need “**real-time**”, think Kinesis.

Streaming Data Examples

- Stock Prices
- Game Data (as the player plays)
- Social Network Data
- Geospatial Data
- Click Stream Data

There are **4** different types of Kinesis Streams



Kinesis Data Streams is a **real-time streaming data service**.

Configure custom Producers and Consumers.

The most flexible data stream option



Amazon (Kinesis) Data Firehose

serverless and a simpler version of Data Streams. Direct integration to specific AWS services.

You pay-on-demand based on how much data consumed.



Managed Service for Apache Flink (*Formally Amazon Kinesis Data Analytics*)

allows you to run queries against data that is flowing through your real-time stream so you can create reports and analysis on emerging data.

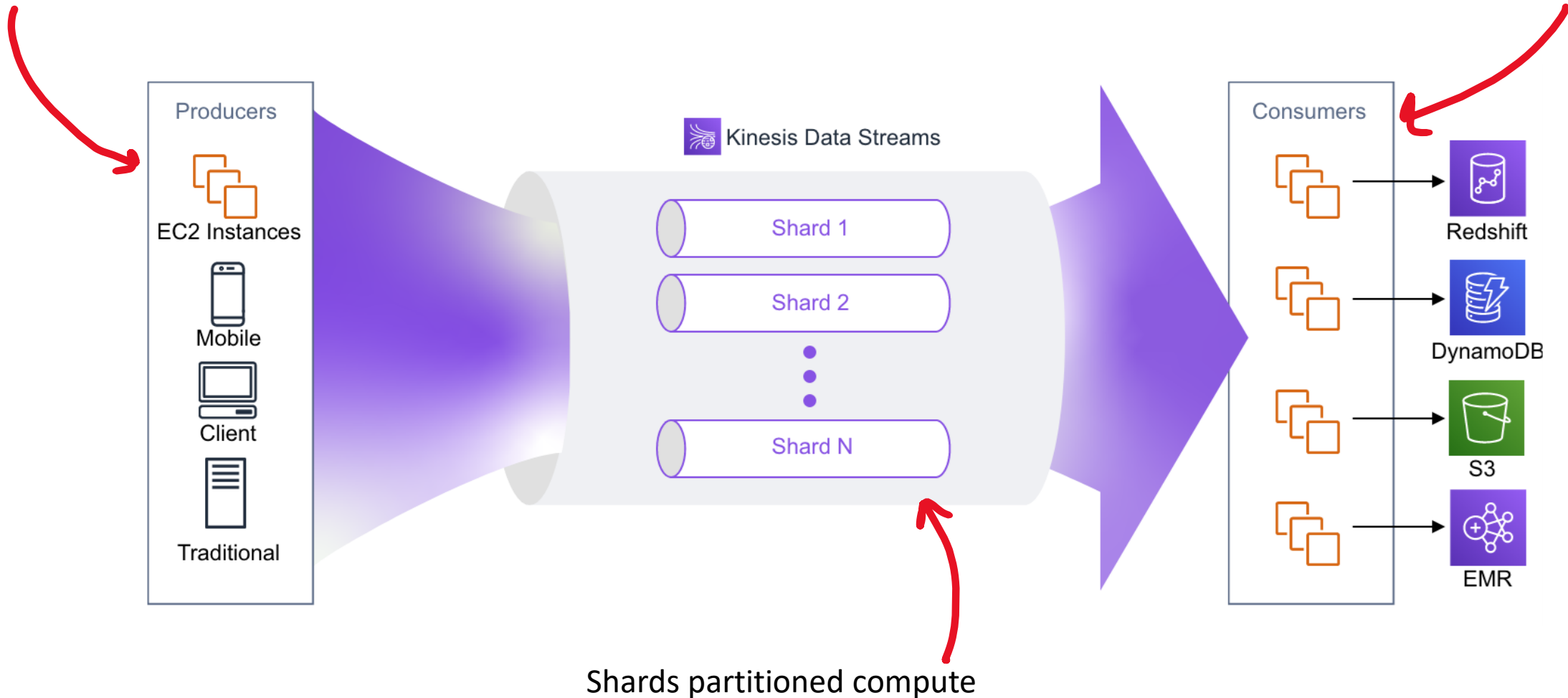


Kinesis Video Streams allows you to analyze or apply processing on real-time streaming video.

Kinesis - Data Streams

Producers (sends data to the stream)

Consumers (receive data from the stream)



Kinesis – Data Streams

Data Streams has two Capacity Modes



Amazon (Kinesis) Data Firehose is a **very similar offering to Kinesis Data Streams On Demand** but Data Firehose is simpler with less customization

	On Demand	Provisioned
Use Case	Unpredictable workloads	Predictable Workloads
Management	Automatically scales	Customer manages shards
Billing	Volume of data ingested and retrieved	Number of shards, data transfer
Capacity	Writes: 200 MiB per second 200,000 records per second Reads: 400 MiB per second per consumer 2 consumers by default Enhanced Fan-Out (EFO) to add 20 more consumers	Write: 1 MiB per second per shard 1,000 records per second per shard Read: 2 MiB per second per shard Max shards is 200

Capacity modes **can be changed at any time** in either direction.

Kinesis – Data Streams– Producers and Consumers

Producers

Amazon Kinesis Agent

Stand alone Java application that will monitor files based on a pattern and send the data to kinesis when it sees changes.

AWS SDK

Use PutRecord, PutRecords for a simple way to publish to a stream. Does not scale.

AWS Direct Integration

Aurora, CloudFront, DynamoDB, Pinpoint, IoT Core...

Amazon Kinesis Producer Library (KPL)

A Java only library that allows you to at scale publish data to a data stream.

Consumers

Third Party

Use other data streams or data processing framework to receive data from data streams:

Apache Flink, Kafka Connect, Adobe Experience, Oracle GoldenGate, Debezium, Fluentd

Kinesis Data Firehose

Send data to Firehose, which in turn directly integrates delivery to other AWS Services.

AWS SDK

Use GetRecords, for a simple way to read from a stream. Does not scale.

Amazon Kinesis Client Library (KCL)

A Java library to write your own custom consumers. MultiLang Daemon allows for other language support

Kinesis – Data Streams Shards

Shards

- A Kinesis data stream is a set of shards.
 - each shard can support up to 5 transactions per second for reads,
 - up to a maximum total data read rate of 2 MB per second
 - up to 1,000 records per second for writes
 - up to a maximum total data write rate of 1 MB per second (including partition keys)
 - each shard has a sequence of data records
 - each data record has a **sequence number** that is assigned by Kinesis Data Stream
 - If your data rate increases, you can increase or decrease the number of shards allocated to your stream

Partition Keys

- used to group data by shard within a stream
- partition keys are Unicode strings, with a maximum length limit of 256 characters for each key
- MD5 hash function is used to map partition keys to 128-bit integer values and to map associated data records to shards using the hash key ranges of the shards
- When an application puts data into a stream, **it must specify a partition key.**

Sequence Number

- Each data record has a sequence number that is unique per partition-key within its shard
- Sequence numbers for the same partition key generally increase over time
 - The longer the time period between write requests, the larger the sequence numbers become

Kinesis – Data Streams Data Retention

Retention period is how long data will remain in the stream until it is released (deleted)

- Data persist for **24 hours** (default)
- Data can be changed to persist for 8760 hours (**365 days**)
 - *Additional charges for retention periods greater than 24 hours*

The retention can be adjusted using the AWS CLI via the the **increase-stream-retention-period** and **decrease-stream-retention-period**.

```
aws kinesis increase-stream-retention-period \  
--stream-name MyStream \  
--retention-period-hours 72
```



It takes several minutes for the retention period to change, and incoming records will follow the previous retention period until the change is complete

Kinesis – Data Streams CLI

Using the AWS CLI the **PutRecord** allows us to send data to the stream.
Note that data has to be base64 encoded



```
echo 'Send reinforcements' | base64
aws kinesis put-record \
  --stream-name $DATA_STREAM_NAME \
  --partition-key $DATA_STREAM_PARTITION_KEY \
  --data U2VuZCByZWluZm9yY2VtZW50cwo=
```

PutRecords can be used for batch records.

Using the AWS CLI **Get-Records** we can retrieve data
Note that you need to pass along the Shard Iterator

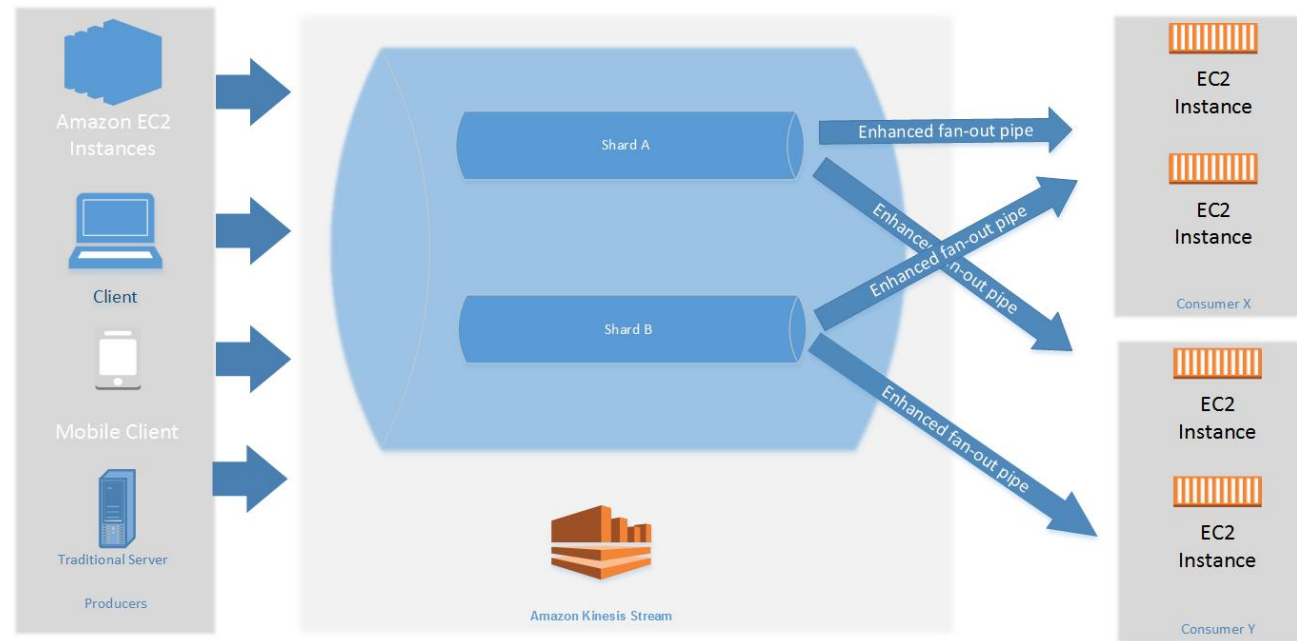


```
export SHARD_ITERATOR=$(aws kinesis get-shard-iterator \
  --shard-id $DATA_STREAM_SHARD \
  --shard-iterator-type TRIM_HORIZON \
  --stream-name $DATA_STREAM_NAME \
  --query 'ShardIterator')
aws kinesis get-records --shard-iterator $SHARD_ITERATOR
```


Kinesis – Enhanced Fan Out (EFO)

Enhanced Fan Out (EFO) allows upto 20 consumers to receive records from a stream with throughput of up to 2 MB of data per second per shard

Consumers that utilize EFO have dedicated throughput per consumer



Consumers must be configured using KCL or Streams API to utilize EFO

Kinesis – KPL

Kinesis Producer Library (KPL) is a managed library by AWS to let you **publish data** to a Kinesis data stream.

<https://github.com/aws-labs/amazon-kinesis-producer>

- When you need to send multiple records per second (mps)
- When you need your producer to vertically scale by 100x
- When you need a producer that is highly efficient of underlying compute resources

KPL is a **java library**.

You have to implement the use of the KPL using Java.



```
public class KPLClickEventsToKinesis extends AbstractClickEventsToKinesis {
    private final KinesisProducer kinesis;

    public KPLClickEventsToKinesis(BlockingQueue<ClickEvent> inputQueue) {
        super(inputQueue);
        kinesis = new KinesisProducer(new KinesisProducerConfiguration()
            .setRegion(REGION)
            .setRecordMaxBufferedTime(5000));
    }

    @Override
    protected void runOnce() throws Exception {
        ClickEvent event = inputQueue.take();
        String partitionKey = event.getSessionId();
        ByteBuffer data = ByteBuffer.wrap(
            event.getPayload().getBytes("UTF-8"));
        while (kinesis.getOutstandingRecordsCount() > 5e4) {
            Thread.sleep(1);
        }
        kinesis.addUserRecord(STREAM_NAME, partitionKey, data);
        recordsPut.getAndIncrement();
    }
}
```

Kinesis – KCL

Kinesis Client Library (KCL) is a java library that makes it easy for developers to easily **consume data** from kinesis

<https://github.com/awslabs/amazon-kinesis-client>

KCL is a java library however via the **MultiLang Daemon** other programming languages such as **Ruby** and **Python** can be used.

```
require 'aws/kclrb'

class SampleRecordProcessor < Aws::KCLrb::V2::RecordProcessorBase
  def init_processor(initialize_input)
    # initialize
  end

  def process_records(process_records_input)
    # process batch of records
  end

  def lease_lost(lease_lost_input)
    # lease was lost, cleanup
  end

  def shard_ended(shard_ended_input)
    # shard has ended, cleanup
  end

  def shutdown_requested(shutdown_requested_input)
    # shutdown has been requested
  end
end

if __FILE__ == $0
  # Start the main processing loop
  record_processor = SampleRecordProcessor.new
  driver = Aws::KCLrb::KCLProcess.new(record_processor)
  driver.run
end
```

```
from amazon_kclpy import kcl
import json, base64

class RecordProcessor(kcl.RecordProcessorBase):

    def initialize(self, initialization_input):
        pass

    def process_records(self, process_records_input):
        pass

    def lease_lost(self, lease_lost_input):
        pass

    def shard_ended(self, shard_ended_input):
        pass

    def shutdown_requested(self, shutdown_requested_input):
        pass

if __name__ == "__main__":
    kclprocess = kcl.KCLProcess(RecordProcessor())
    kclprocess.run()
```

Amazon Data Firehose



Amazon Data Firehose (*formally Kinesis Firehose Delivery Systems*) allows for simple transformation and delivery of data



- You choose **one consumer** from a predefined list
- Data **immediately disappears** once it's consumed
- You can convert incoming data to a few other file formats and compress and secure data
- You pay only for data that is ingested

Amazon Data Firehose – Sources

Data Firehose allows easily configure a source or *sources of data often with minimal or no programming skills.

You aren't forced to learn Java to best leverage Firehose like Kinesis Data Streams

```
require 'aws-sdk-firehose'

# Initialize a Kinesis client
client = Aws::Firehose::Client.new

stream_name = 'My-Firehose'

# Prepare records
10.times.map do |i|
  data = {hello: "world: #{i}"}.to_json
  response = client.put_record(
    delivery_stream_name: stream_name,
    record: {data: data}
  )
  #binding.pry
  puts "Response: #{response.inspect}"
end
```

Direct Put SDK using Ruby

Source

1. Data Streams can send to firehose
2. Amazon Managed Streaming for Apache Kafka (MSK)
 - Destination can only be S3
3. Direct PUT (many supported services or use the SDK/CLI)

- AWS SDK
- AWS Lambda
- AWS CloudWatch Logs
- AWS CloudWatch Events
- AWS Cloud Metric Streams
- AWS IOT
- AWS Eventbridge
- Amazon Simple Email Service
- Amazon SNS
- AWS WAF web ACL logs
- Snowflake
- Apache Nifi

- Amazon API Gateway - Access logs
- Amazon Pinpoint
- Amazon MSK Broker Logs
- Amazon Route 53 Resolver query logs
- AWS Network Firewall Alerts Logs
- AWS Network Firewall Flow Logs
- Amazon Elasticache Redis SLOWLOG
- Kinesis Agent (linux)
- Kinesis Tap (windows)
- Fluentbit
- Fluentd

Amazon Data Firehose – Destinations

Data Firehose can send to specific AWS services, third party services or an HTTP Endpoint

AWS Services Destinations

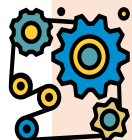
- **Amazon S3**
- **Amazon Redshift**
- Amazon OpenSearch Service
- Amazon OpenSearch Serverless

Custom Destinations

- HTTP Endpoint

Third Party Destinations

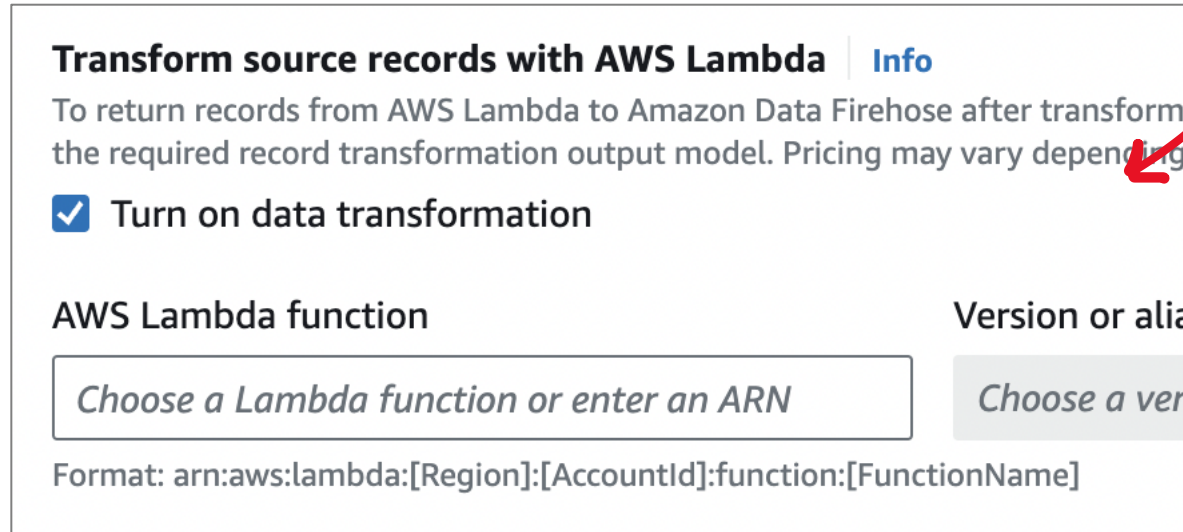
- Coralogix
- Datadog
- Dynatrace
- Elastic
- Honeycomb
- Logic Monitor
- Logz.io
- MongoDB Cloud
- New Relic
- Splunk
- Splunk Observability Cloud
- Sumo Logic
- Snowflake



Each destination has different destination configuration options, often third-party apps need an API key from the provider

Amazon Data Firehose – Data Transformation

Before data is sent to a destination it can be **transformed** with AWS Lambda



Transform source records with AWS Lambda | [Info](#)

To return records from AWS Lambda to Amazon Data Firehose after transforming the required record transformation output model. Pricing may vary depending on the configuration.

☒ Turn on data transformation

AWS Lambda function Version or alias

Format: arn:aws:lambda:[Region]:[AccountId]:function:[FunctionName]

When enabled Firehose will buffer incoming data.

- You can change the buffer size: 0.2 to 3MB
- You can change the buffer interval 0-900 seconds.

There is a **payload size limit** of 6MB for both the Lambda request and the response

Amazon Data Firehose – Data Transformation

When transforming data you need to ensure you return a json payload with the following:

- RecordId – the original record id passed from Firehose
- Result:
 - Ok
 - Dropped
 - Processing Failed
- Data
 - The transformed data base64 encoded

```
import base64
def lambda_handler(event, context):
    output = []

    for record in event['records']:
        print(record['recordId'])
        payload = base64.b64decode(record['data']).decode('utf-8')

        # Do custom processing on the payload here

        output_record = {
            'recordId': record['recordId'],
            'result': 'Ok',
            'data': base64.b64encode(payload.encode('utf-8')).decode('utf-8')
        }
        output.append(output_record)

    print('Successfully processed {} records.'.format(len(event['records'])))

    return {'records': output}
```

AWS Lambda has a **blueprint** for transforming data via lambda.

Amazon Data Firehose – Dynamic Partitioning

Dynamic partitioning enables you to **continuously partition streaming data** in Firehose by using keys within data and then deliver the **data grouped by these keys into Amazon S3 Prefixes**

This makes it easier to run high performance, cost-efficient analytics on streaming data in Amazon S3 using various services such:

- Amazon Athena
- Amazon EMR
- Amazon Redshift Spectrum
- Amazon QuickSight

This makes it easier for AWS Glue can perform more sophisticated extract, transform, and load (ETL) jobs

To select the key and values for partitioning you have two ways:

- Inline Partitioning — You provide a JQ expression to parse json
- Lambda function — You have a lambda parse the data and return the keys and values



Once enabled you cannot turn off Dynamic Partitioning on a Firehose stream

Amazon Data Firehose – Dynamic Partitioning

If you have the following
json data payload

```
{
  "type": {
    "device": "mobile",
    "event": "user_clicked_submit_button"
  },
  "customer_id": "1234567890",
  "event_timestamp": 1565382027,
  "region": "pdx"
}
```

With inline you add they
keys and JQ expressions

Key name	JQ expression	
customer_id	.customer_id	Remove
device	.type.device	Remove
year	.event_timestamp strftime("%Y")	Remove
<button>Add dynamic partitioning key</button>		

Amazon Data Firehose – Convert Record Format

Data Firehose can convert JSON data into different file formats before being delivered to S3

- Apache Parquet
- Apache ORC



Convert record format [Info](#)

Data in Apache Parquet or Apache ORC formatted source records using a schema function that converts them to JSON in

☒ Enable record format conversion

Output format

☒ Apache Parquet

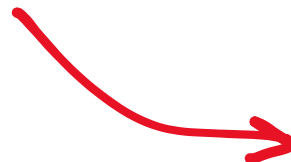
☐ Apache ORC



AWS Glue table is used to specify the schema of your source records for Convert Record Format

Amazon Data Firehose – Compression

You can have data records compressed before be delivered to your S3 Bucket



Compression for data records

Amazon Data Firehose can compress records before they are delivered to your S3 bucket.

☒ Not enabled

☐ GZIP

☐ Snappy

☐ Zip

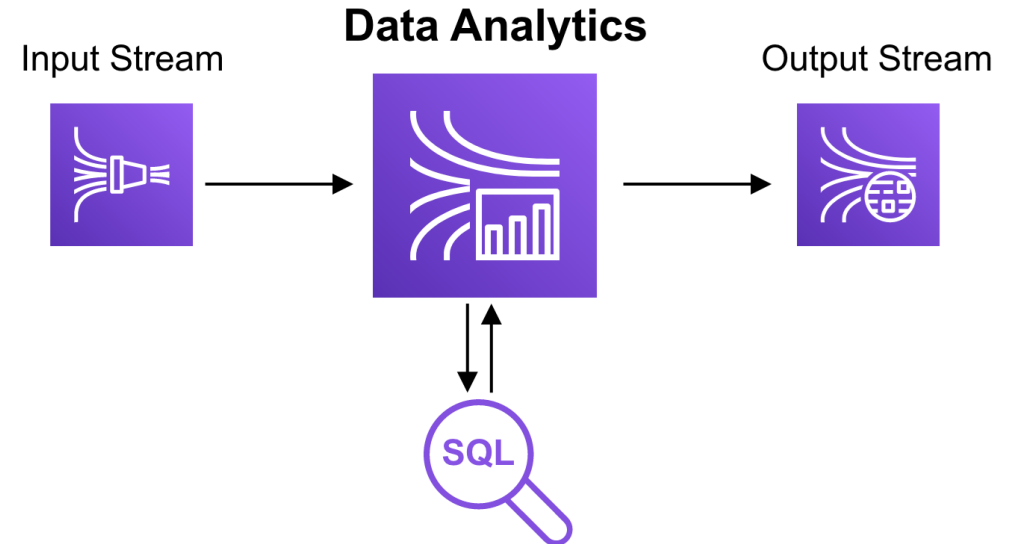
☐ Hadoop-Compatible Snappy

Managed Service for Apache Flink



Managed Service for Apache Flink (*Formally Amazon Kinesis Data Analytics*) allows you to run queries against data that is flowing through your real-time stream so you can create reports and analysis on emerging data.

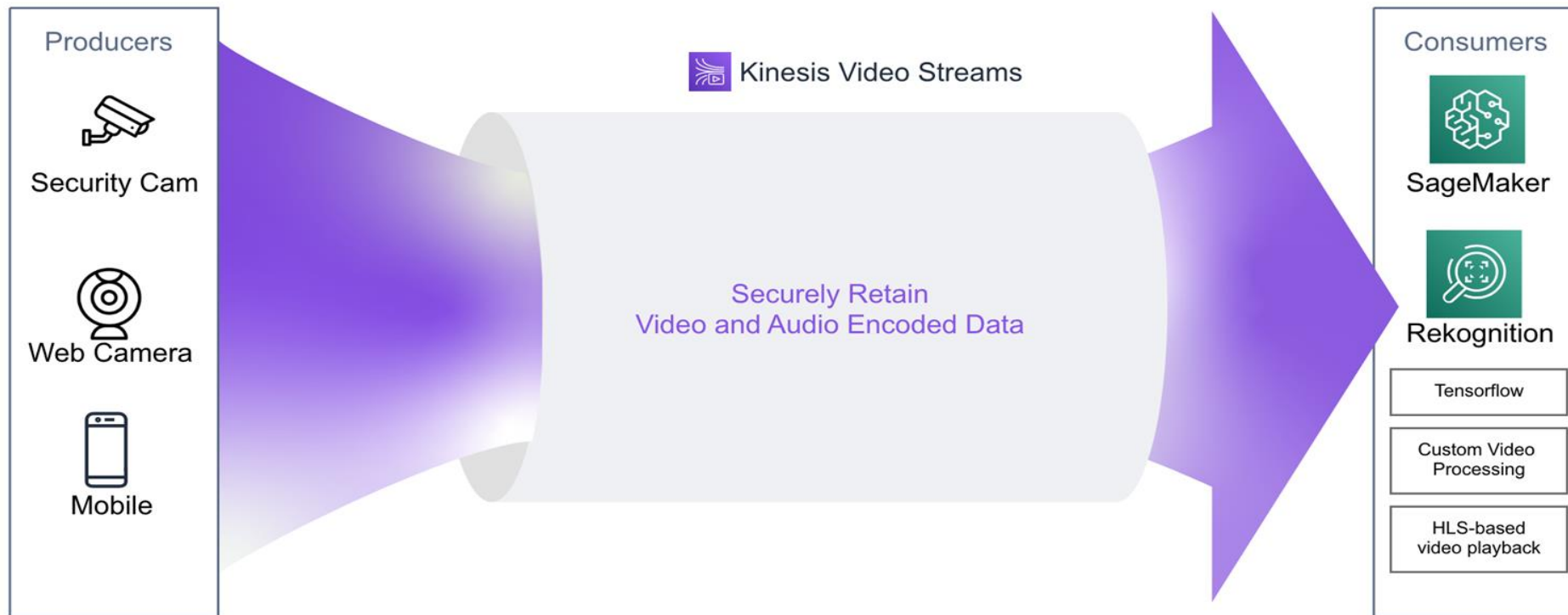
- You can get specific Firehose or Data Streams as an input and an output.
- Data that pass through Data Analytics is run through **custom SQL** you provide, and the results are then output.



Kinesis Video Streams



Kinesis Video Streams a fully managed AWS service, to stream live video from devices to the AWS Cloud, or build applications for real-time video processing or batch-oriented video analytics.



- Ingest video and audio-encoded data from various devices and/or services.
- Output video data to ML or video processing services